



PRINCIPI DELLA PROGRAMMAZIONE PARALLELA · A.A. 2025/2026

Simulazione della diffusione del Calore 2D

Simulazione parallela ibrida con MPI + OpenMP

Stencil a 5 punti

Decomposizione del dominio

Strong / Weak scaling

INDICE



1 PRESENTAZIONE DEL PROBLEMA

2 INPUT

3 MODELLO ARCHITETTURALE

4 OUTPUT

5 RISULTATI

6 CONCLUSIONI

PRESENTAZIONE DEL PROBLEMA



CONTESTO SCIENTIFICO

La simulazione numerica rappresenta il "terzo pilastro" della scienza moderna, affiancando l'analisi teorica e quella sperimentale.

OBIETTIVO DEL PROGETTO

Studiare l'evoluzione termica nel tempo di una piastra bidimensionale attraverso un simulatore flessibile e configurabile.

IL CASO DI TEST

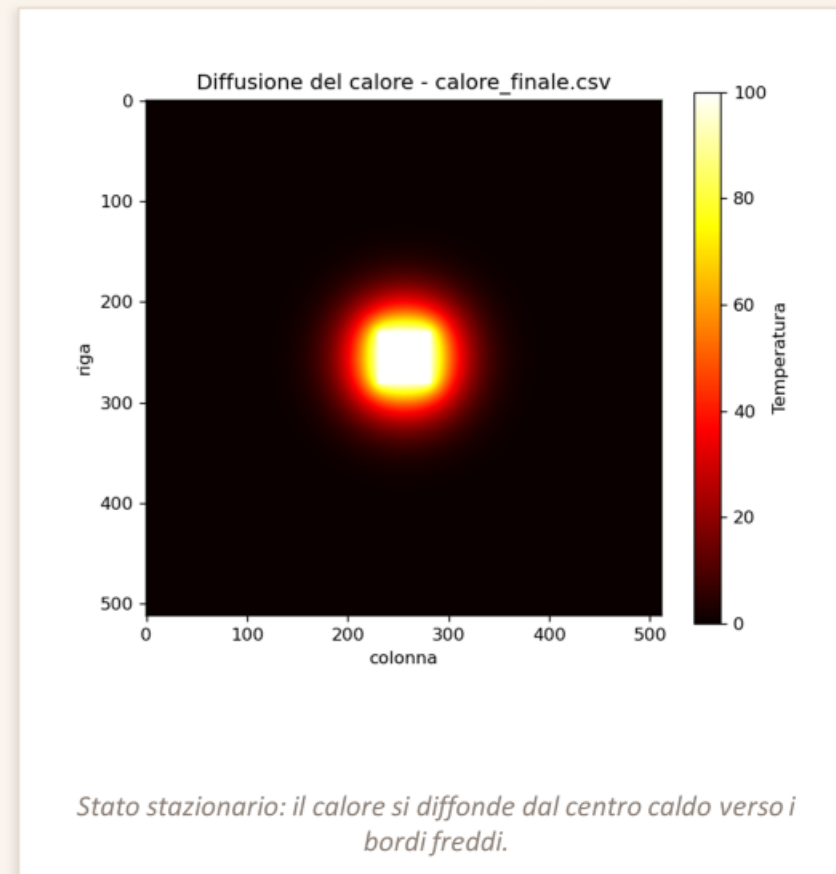
Una piastra rettangolare con i bordi tenuti freddi ($T = 0$) e un oggetto caldo fisso al centro ($T = 100$).

LA SCALABILITA'

Su griglie di grandi dimensioni, il costo computazionale e la memoria richiesta crescono rapidamente, rendendo insufficiente una singola macchina

LA SOLUZIONE

Utilizzo di una strategia ibrida per una parallelizzazione interna ed esterna





Equazione del calore e stencil a 5 punti

EQUAZIONE DEL CALORE

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

DISCRETIZZAZIONE DELLA FORMULA

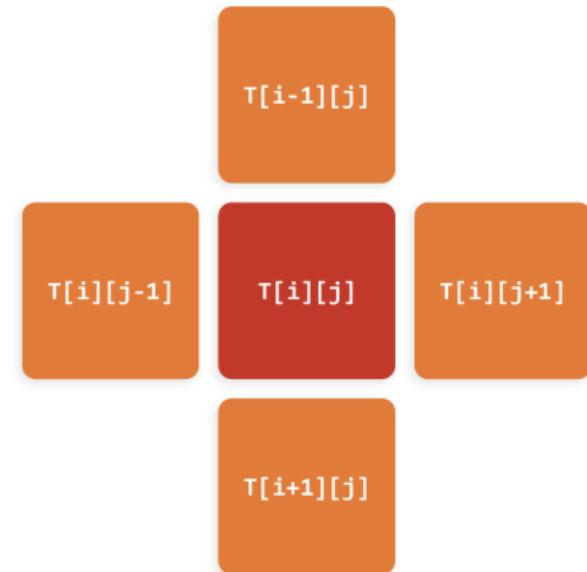
$$T[i][j] += \alpha \cdot (T[i+1][j] + T[i-1][j] + T[i][j+1] + T[i][j-1] - 4 \cdot T[i][j])$$

ACCURATEZZA SPAZIALE

Approssimazione della derivate seconda con differenze centrali e errore di troncamento $O(h^2)$

SCHEMA ESPPLICITO

Il calcolo dello stato futuro dipende esclusivamente dal passo precedente, rendendo l'algoritmo ideale per la parallelizzazione



La nuova temperatura di una cella dipende dai 4 vicini.

INPUT



Il progetto è stato sviluppato per garantire un'elevata flessibilità, consentendo di configurare i parametri geometrici del dominio e le condizioni fisiche direttamente da riga di comando

```
OMP_NUM_THREADS=4 mpirun -np $(NP)
./diffusione $(NX) $(NY) $(max_iter)
--diffusion $(ALPHA)
--temp-edges $(TEMP_EDGES)
--temp-source $(TEMP_SOURCE)
--save-each $(SAVE_EACH)
```

Parametro	Descrizione
<code>NX</code>	Numero di righe della griglia.
<code>NY</code>	Numero di colonne della griglia.
<code>max_iter</code>	Numero massimo di iterazioni temporali.
<code>alpha</code>	Coefficiente di diffusione (default: 0.25 per garantire stabilità numerica).
<code>temp_edges</code>	Temperatura dei bordi della griglia (default: 0)
<code>temp_source</code>	Temperatura della sorgente della griglia (default: 100). Essendo un corpo caldo, la temperatura della sorgente deve essere maggiore della temperatura dei bordi.
<code>save_each</code>	Frequenza di salvataggio dello stato della griglia.

MODELLO ARCHITETTURALE



Decomposizione del dominio

`MPI`

Ogni processo elabora le sue righe ($NX/size$) e memorizza due righe fantasma (halo) per comunicare con i vicini.



Halo exchange

`MPI_Sendrecv`

Ogni passo i processi si scambiano le righe di bordo con i vicini. Sendrecv evita il deadlock delle Send bloccanti; `MPI_PROC_NULL` gestisce i bordi.



Calcolo locale

`#pragma omp parallel for`

Le righe locali sono distribuite tra i thread OpenMP. `reduction(max:)` calcola l'errore massimo senza race condition.



Convergenza

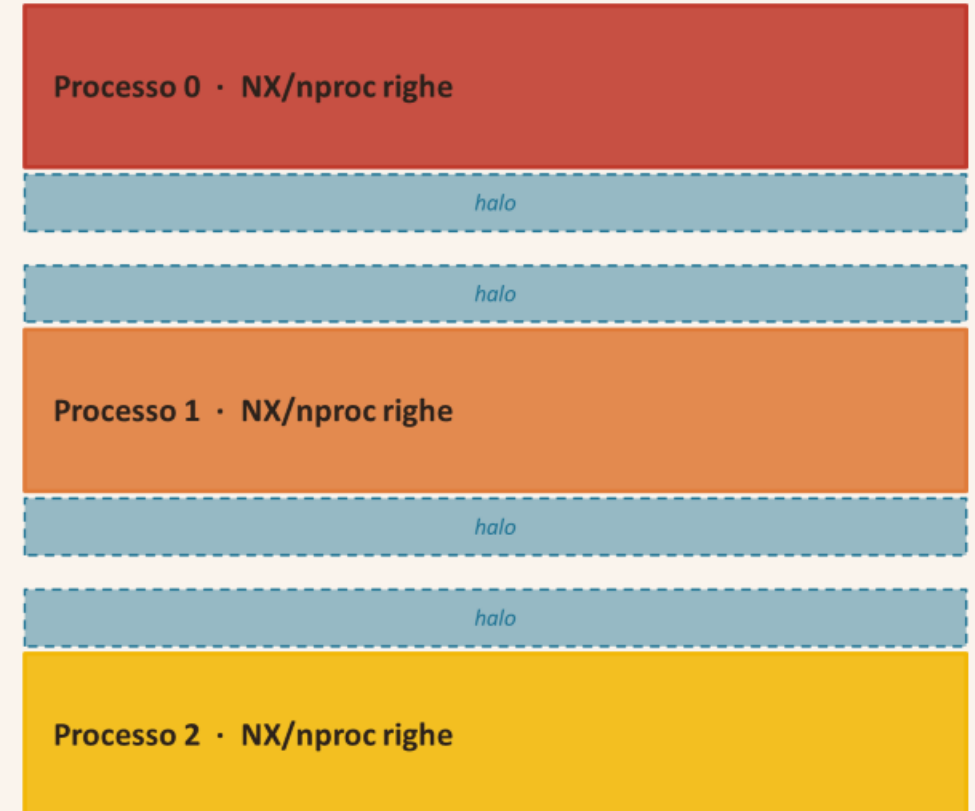
`MPI_Allreduce(MPI_MAX)`

Si combina l'errore massimo di tutti i processi. Tutti ricevono il risultato e decidono insieme se fermarsi. Più efficiente di Reduce + Bcast.



Decomposizione del dominio e righe “fantasma”

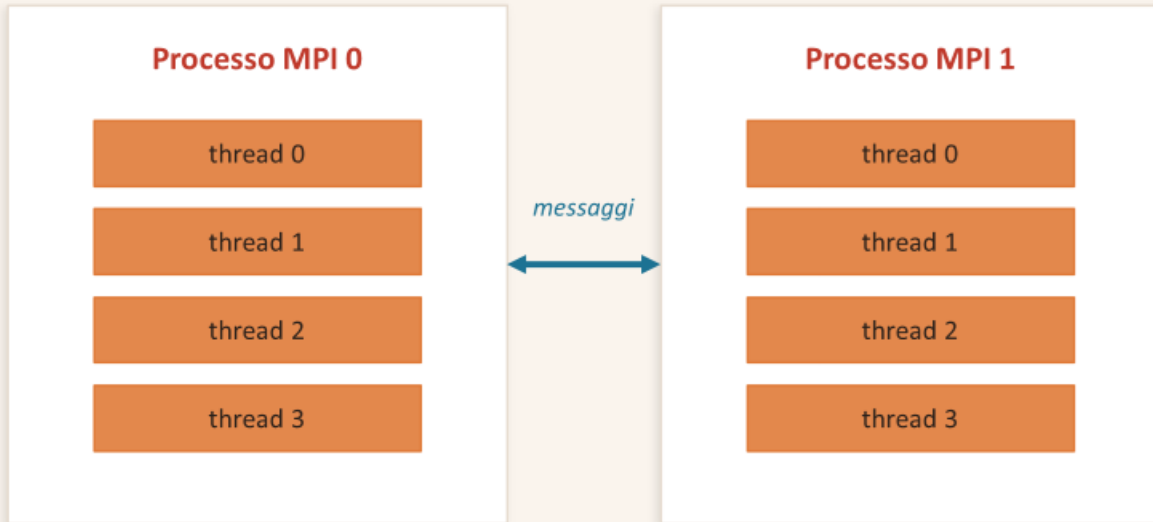
- La griglia è divisa in strisce orizzontali: una per processo MPI.
- Ogni processo gestisce $NX / nproc$ righe consecutive.
- Grazie al layout *row-major*, l'accesso ai dati è perfettamente contiguo: l'assenza di salti in memoria (*stride*) massimizza la località spaziale.
- **Servono 2 righe “fantasma” (halo)** Copia delle righe di bordo dei processi vicini, necessarie allo stencil.
- Ai bordi globali il vicino è `MPI_PROC_NULL` al fine di disabilitare lo scambio



Griglia globale $NX \times NY$ divisa a strisce



MPI tra i processi, OpenMP dentro al processo



Memoria distribuita (MPI) + memoria condivisa (OpenMP)

pseudocodice del ciclo nel tempo

```
per iter = 1 .. max_iter:  
    # 1) scambio bordi coi vicini  
    MPI_Sendrecv(sopra/sotto)  
  
    # 2) calcolo (multithread)  
    #pragma omp parallel for  
    for righe locali: stencil 5 punti  
  
    # 3) errore globale  
    MPI_Allreduce(MAX)  
  
    scambia T <-> T_nuovo  
    se errore < tolleranza: stop
```


|OUTPUT

Al termine della simulazione, o a intervalli temporali preconfigurati, le porzioni locali della griglia vengono raccolte dai diversi processi MPI per ricostruire la configurazione completa del dominio tramite `MPI_Gather()`. I dati così consolidati, fondamentali per le successive fasi di validazione matematica e visualizzazione grafica, vengono salvati su disco in formato testuale.

Parametro	Descrizione
<code>T[i][j]</code>	Griglia di temperatura salvata in CSV ogni N iterazioni visualizzabile con Python/matplotlib.
<code>error_iter</code>	Errore di convergenza in funzione delle iterazioni
<code>n_iter</code>	Numero di iterazioni necessarie alla convergenza



DATI, ...

tempi e MUPS per i grafici



convergenza.csv

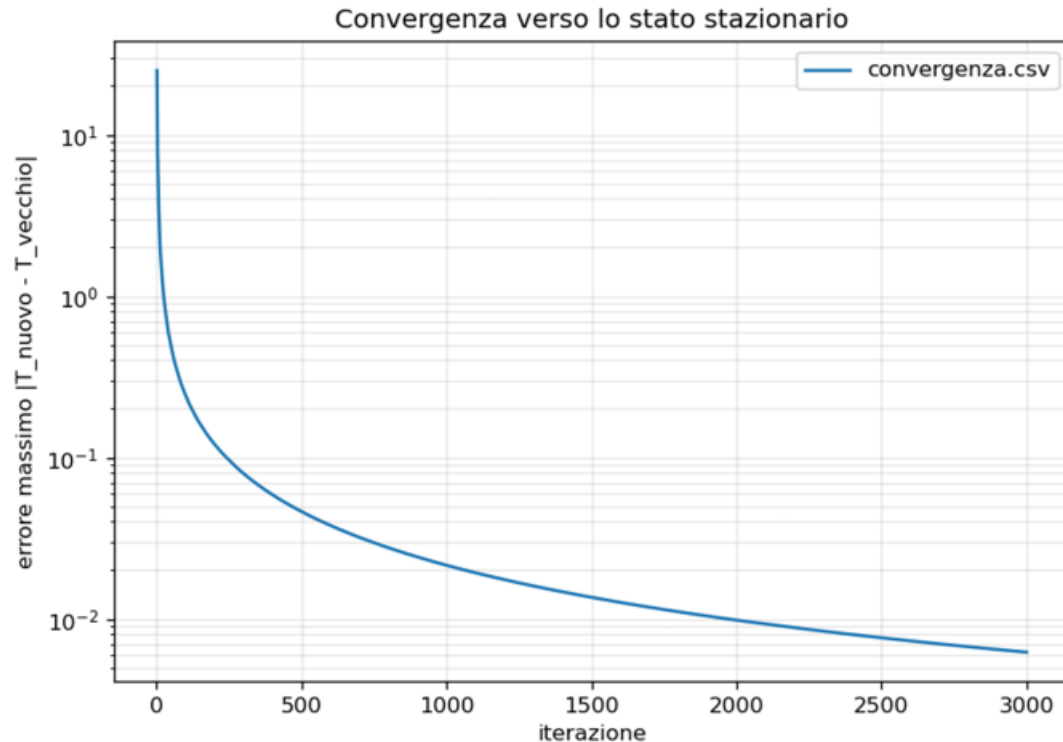
errore per iterazione



calore_finale.csv

mappa di temperatura

Convergenza verso lo stato stazionario



CONVERGENZA ALLO STATO STAZIONARIO

La variazione massima della temperatura tra due passi temporali ($\max |\Delta T|$) decresce in modo monotono verso lo zero, indicando il raggiungimento dell'equilibrio termico.

IMPATTO DELLA RISOLUZIONE

All'aumentare delle dimensioni della griglia spaziale, cresce il numero di iterazioni necessarie per propagare il calore e raggiungere la convergenza.

EFFETTO DEL PARAMETRO α

Il valore critico $\alpha = 0.25$ garantisce il massimo passo temporale consentito, minimizzando le iterazioni. Valori subcritici ($\alpha < 0.25$) mantengono la stabilità, ma rallentano l'avanzamento della simulazione, richiedendo un numero maggiore di passi per raggiungere lo stato stazionario.

Modello di costo e leggi di scalabilità



Costo della comunicazione (α - β)

$$T(n) = \alpha + n / \beta$$

α = latenza (costo fisso del messaggio), β = banda

OTTIMIZZAZIONE DEI MESSAGGI

Ogni invio ha un costo di partenza fisso (α). Per questo è più efficiente impacchettare i dati in pochi messaggi grandi anziché frammentarli.

RAPPORTO CALCOLO-COMUNICAZIONE

Nel dominio 2D, il calcolo scala con l'area del dominio $O(N^2)$, mentre lo scambio dei bordi (halo) scala con il perimetro $O(N)$. Di conseguenza, per problemi di grandi dimensioni, il peso percentuale della comunicazione diventa marginale.



Limiti dello speedup

Amdahl (strong)

$$S = 1 / (s + (1-s)/p)$$

Problema fisso: limite $1/s$.

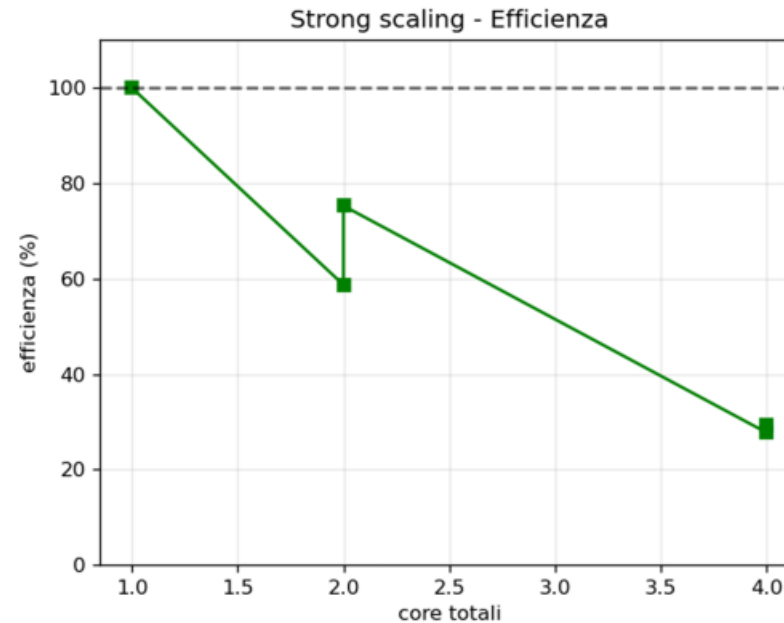
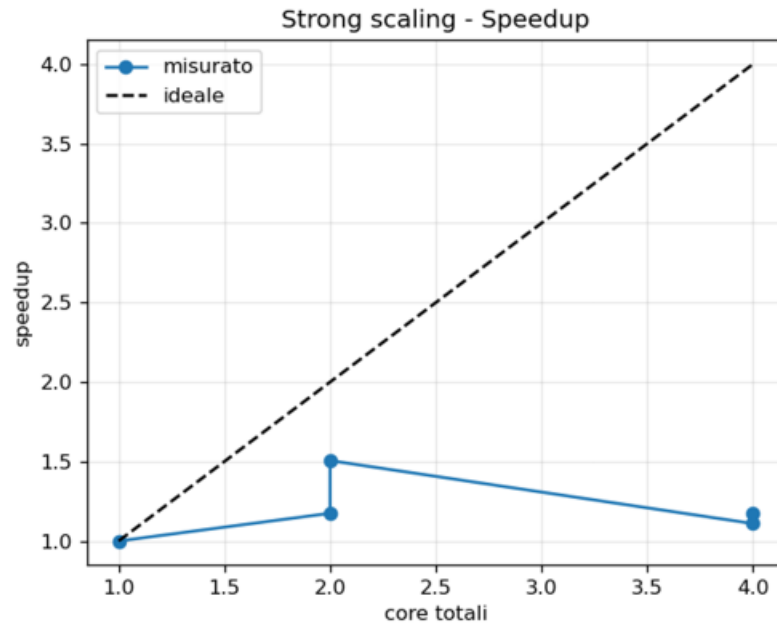
Gustafson (weak)

$$S = p - s \cdot (p-1)$$

Problema che cresce: nessun limite rigido.

La “parte seriale” in MPI include comunicazione, sincronizzazione e sbilanciamento di carico, non solo il codice sequenziale.

Strong scaling (griglia fissa 1024^2)



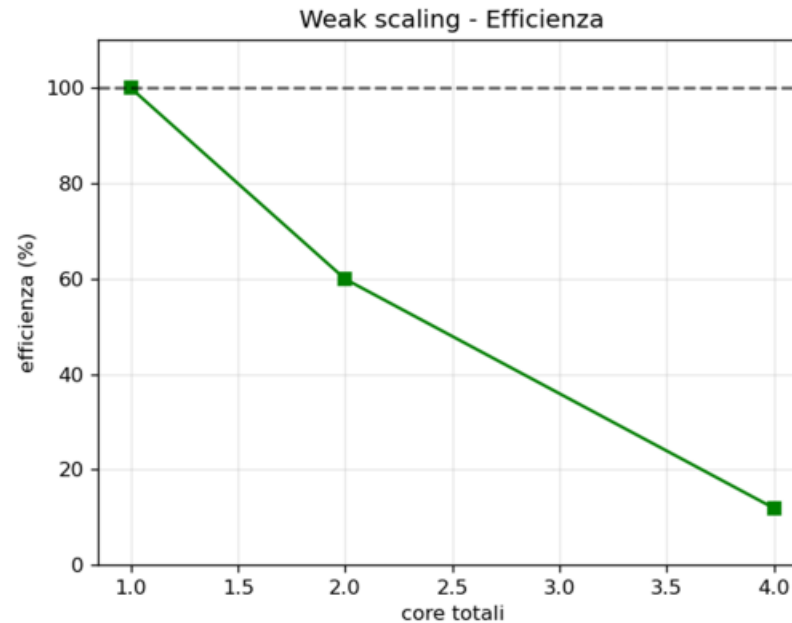
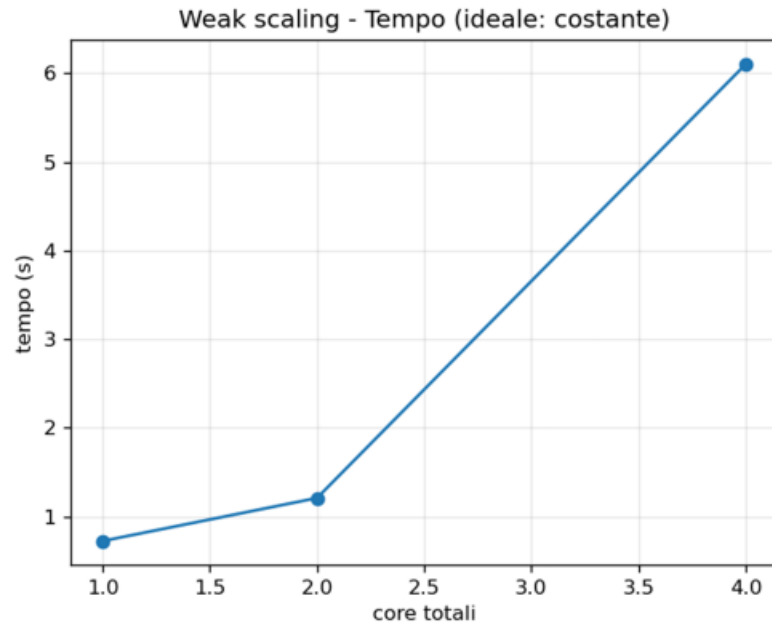
Cosa succede

- Griglia fissa, aumento i core: lo speedup cresce ma resta lontano dall'ideale.
- A 2 core: OpenMP (1×2) = 75% > MPI (2×1) = 59%.
- OpenMP intra-nodo non paga halo né Allreduce.
- A 4 core: efficienza ~28% (crollo).



Coerente con Amdahl: a griglia fissa la comunicazione pesa di più man mano che il calcolo per processo si riduce.

Weak scaling (512 righe per processo)



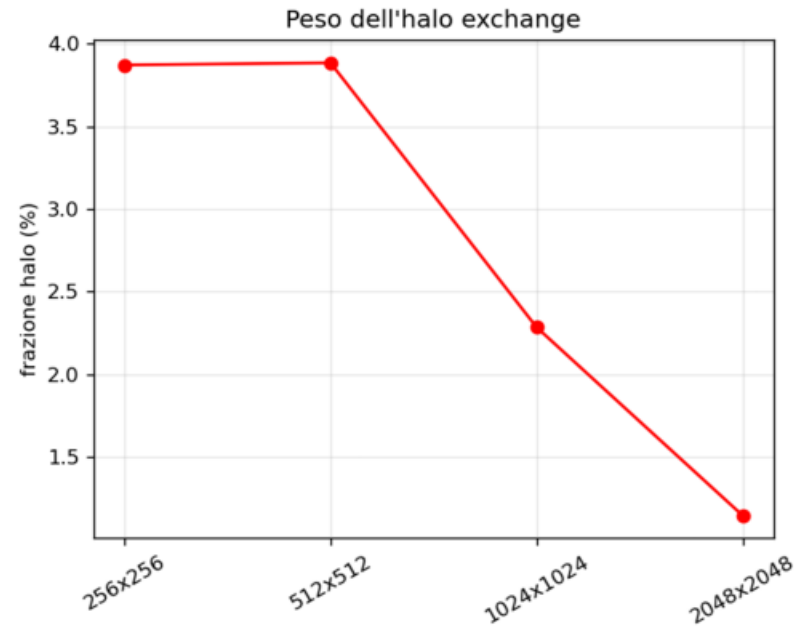
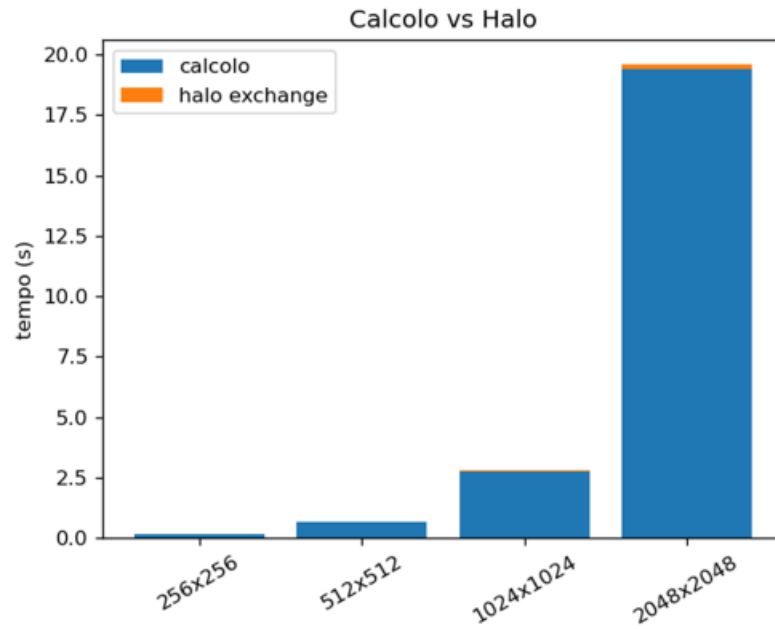
Cosa succede

- 2 processi → efficienza ~60%.
- 4 processi → crollo al 12% (tempo da 0.7 s a 6.1 s).
- Segno di oversubscription: pochi core fisici.
- Fino a 2 processi il comportamento è ragionevole; oltre, il limite è l'hardware (≈2 core fisici), non il codice.



Coerente con Gustafson: incrementando le risorse computazionali proporzionalmente alla dimensione del problema, il tempo di esecuzione dovrebbe mantenersi asintoticamente costante.

Halo exchange vs calcolo



Cosa succede

- Più la griglia cresce, meno pesa l'halo: dal 3.9% all'1.1%.
- **Calcolo $\sim O(N^2)$, halo $\sim O(N)$.**
- Rapporto superficie/volume favorevole sui problemi grandi.
- In ogni caso il calcolo domina nettamente.



La comunicazione **NON** è il collo di bottiglia: lo sono il calcolo (memory-bound) e l'hardware locale.

Cosa ci dicono i numeri

472

MUPS

picco (1 proc × 2 thread)

75%

Efficienza

massima, a 2 core (OpenMP)

1.1%

Halo Time

su griglia 2048² (era 3.9%)

≈ 2

Core fisici

oltre questi: oversubscription

- Il parallelismo dà beneficio reale fino a 2 core su questa macchina.
- A parità di core, OpenMP intra-nodo rende più di MPI (no messaggi).
- Il kernel è memory-bound: contano località e banda di memoria, non l'halo.

CONCLUSIONI



VALIDAZIONE DEL MODELLO

L'implementazione ibrida MPI + OpenMP è robusta e corretta: i risultati numerici finali sono rigorosamente invarianti rispetto alla combinazione di processi e thread scelti.

ARCHITETTURA EFFICIENTE

: La decomposizione 1D a strisce, unita all'uso di MPI_Sendrecv per i bordi e MPI_Allreduce per il controllo della convergenza, garantisce un'ottima sinergia tra memoria distribuita e condivisa.

DINAMICHE DI SCALING

I test confermano i modelli teorici: lo *strong scaling* è fisiologicamente limitato dalle comunicazioni e dalla frazione seriale (Legge di Amdahl), mentre il *weak scaling* mostra un'eccellente scalabilità (Legge di Gustafson).

ANALISI DEI LIMITI

Sui problemi di grandi dimensioni, l'overhead di rete (halo exchange) diventa marginale. Il vero collo di bottiglia è l'hardware locale, a causa della natura intrinsecamente *memory-bound* dell'algoritmo e della limitata disponibilità di core fisici



GRAZIE PER
L'ATTENZIONE